

Nama : Elyza Putri Rahayu

NIM : 254107020035

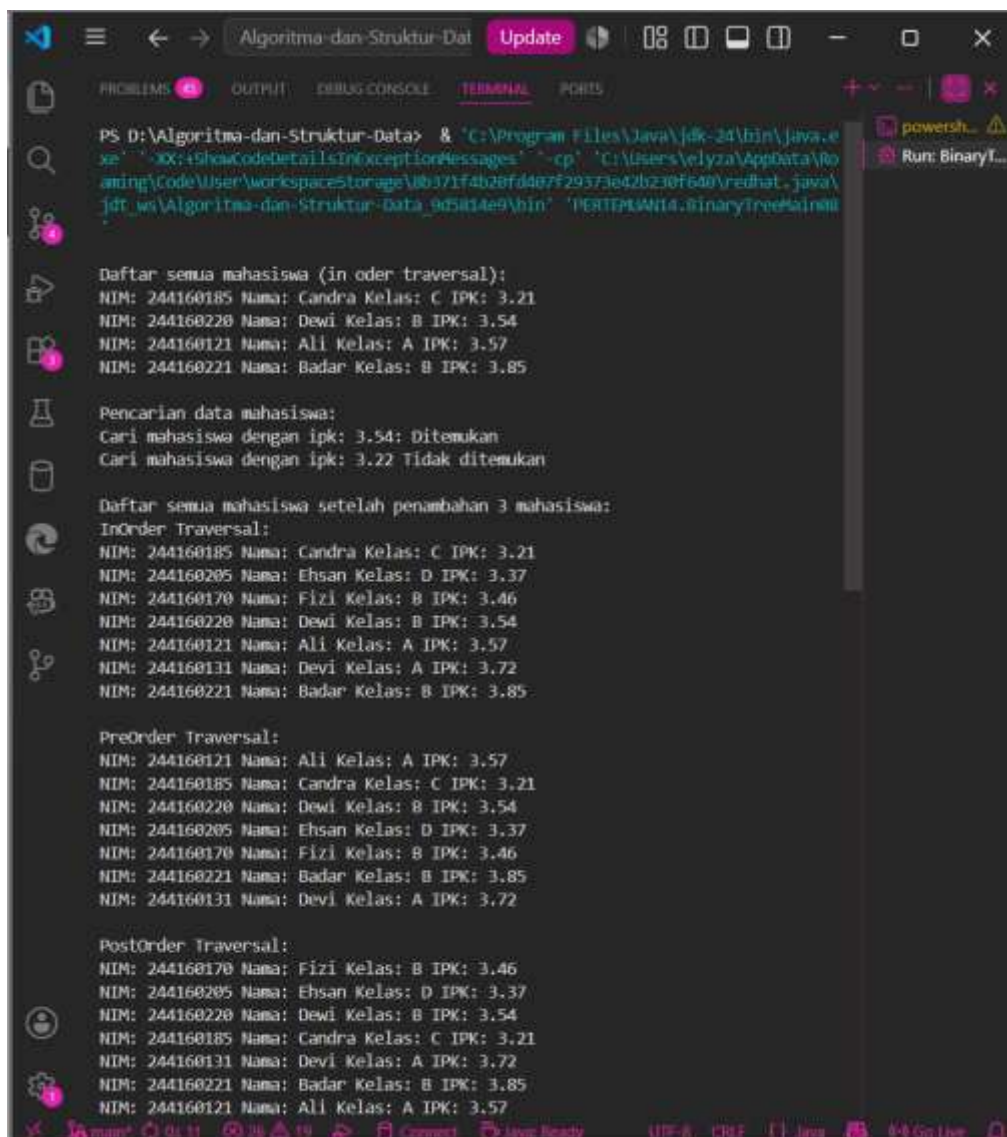
Kelas : TI – 1G

No Absen : 08

LAPORAN JOBSHEET 14

PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

PERCOBAAN 1



```
PS D:\Algoritma-dan-Struktur-Dat> & 'C:\Program Files\Java\jdk-24\bin\java.exe' ^
  -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\elyza\AppData\Roaming\Code\User\workspacestorage\0b371fab28fd487f29373e42b230f6d0\redhat-java\jdt_ws\Algoritma-dan-Struktur-Dat_9d5814e9\bin' 'PERTEMUAN14.BinaryTreeMain'

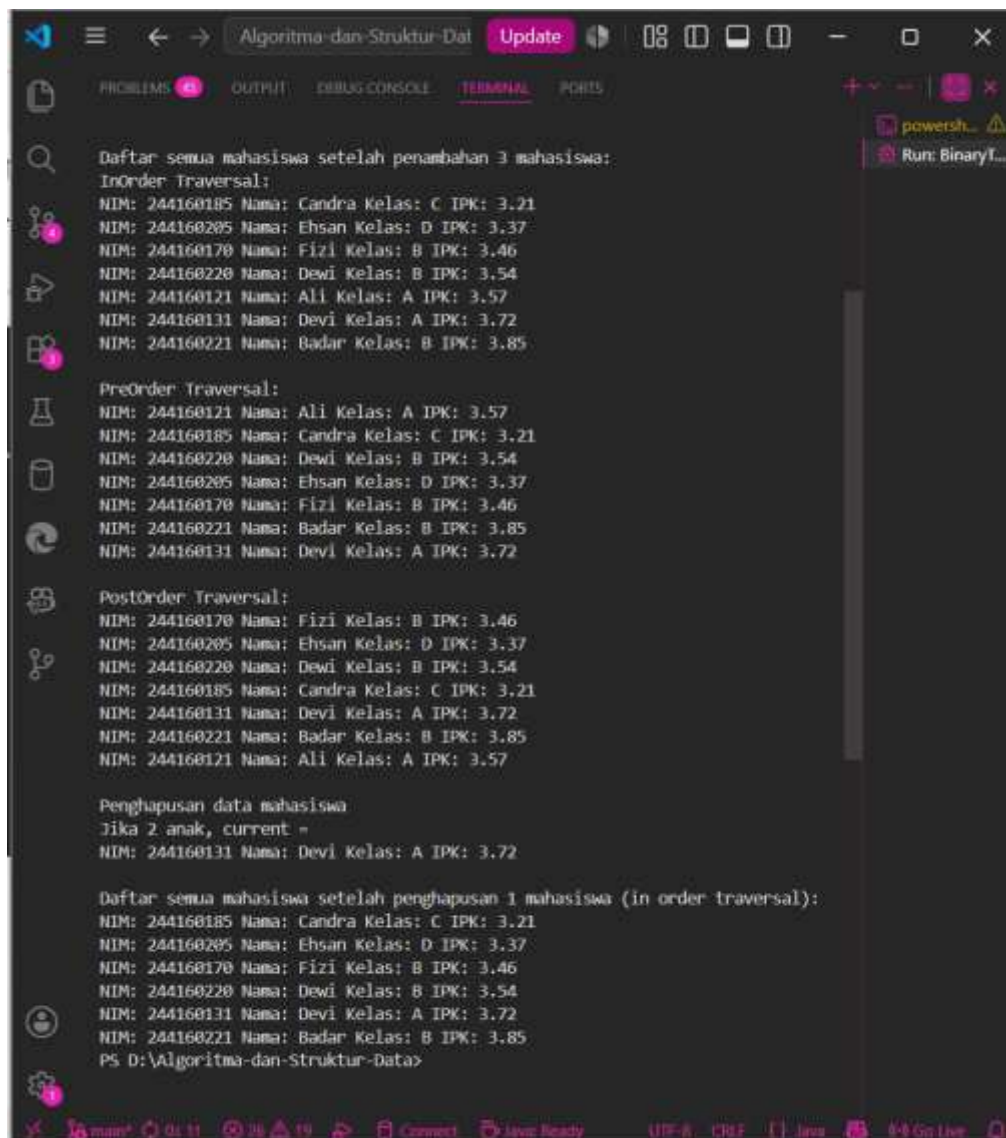
Daftar semua mahasiswa (in oder traversal):
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

Pencarian data mahasiswa:
Cari mahasiswa dengan ipk: 3.54: Ditemukan
Cari mahasiswa dengan ipk: 3.22 Tidak ditemukan

Daftar semua mahasiswa setelah penambahan 3 mahasiswa:
InOrder Traversal:
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

PreOrder Traversal:
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160131 Nama: Ali Kelas: A IPK: 3.72

PostOrder Traversal:
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
```



```
Algoritma-dan-Struktur-Dat Update
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Daftar semua mahasiswa setelah penambahan 3 mahasiswa:
InOrder Traversal:
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

PreOrder Traversal:
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72

PostOrder Traversal:
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57

Penghapusan data mahasiswa
Jika 2 anak, current =
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
PS D:\Algoritma-dan-Struktur-Data>
```

Kode Program Mahasiswa08 :

```
package PERTEMUAN14;
```

```
public class Mahasiswa08 {
```

```
    String nim;
```

```
    String nama;
```

```
    String kelas;
```

```
    double ipk;
```

```
    public Mahasiswa08 () {
```

```
}
```

```
public Mahasiswa08(String nim, String nama, String kelas, double ipk) {
```

```
    this.nim = nim;
```

```
    this.nama = nama;
```

```
    this.kelas = kelas;
```

```
    this.ipk = ipk;
```

```
}
```

```
public void tampilInformasi() {
```

```
    System.out.println("NIM: "+this.nim+" " + "Nama: "+this.nama+" "+"Kelas:  
"+this.kelas+" "+"IPK: "+this.ipk);
```

```
}
```

```
}
```

Kode Program Node08 :

```
package PERTEMUAN14;
```

```
public class Node08 {
```

```
    Mahasiswa08 mahasiswa;
```

```
    Node08 left, right;
```

```
public Node08() {
```

```
}
```

```
public Node08(Mahasiswa08 mahasiswa) {
```

```
    this.mahasiswa = mahasiswa;
```

```
    left = right = null;
```

```
}
```

```
}
```

Kode Program BinaryTree08 :

```
package PERTEMUAN14;
```

```
public class BinaryTree08 {
```

```
    Node08 root;
```

```
    public BinaryTree08() {
```

```
        root = null;
```

```
    }
```

```
    public boolean isEmpty() {
```

```
        return root == null;
```

```
    }
```

```
    public void add(Mahasiswa08 mahasiswa) {
```

```
        Node08 newNode08 = new Node08(mahasiswa);
```

```
        if (isEmpty()) {
```

```
            root = newNode08;
```

```
        } else {
```

```
            Node08 current = root;
```

```
            Node08 parent = null;
```

```
            while (true) {
```

```
                parent = current;
```

```
                if (mahasiswa.ipk < current.mahasiswa.ipk) {
```

```
                    current = current.left;
```

```
                    if (current == null) {
```

```
                        parent.left = newNode08;
```

```
                        return;
```

```
                    }
```

```

    } else {
        current = current.right;
        if (current == null) {
            parent.right = newNode08;
            return;
        }
    }
}
}
}
}
}

```

```

boolean find(double ipk) {
    boolean result = false;
    Node08 current = root;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            result = true;
            break;
        } else if (ipk > current.mahasiswa.ipk) {
            current = current.right;
        } else {
            current = current.left;
        }
    }

    return result;
}

```

```

void traversePreOrder (Node08 node) {
    if (node != null) {

```

```

        node.mahasiswa.tampilInformasi();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}

```

```

void traverseInOrder(Node08 node) {
    if (node != null) {
        traverseInOrder(node.left);
        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}

```

```

void traversePostOrder(Node08 node) {
    if (node != null) {
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.mahasiswa.tampilInformasi();
    }
}

```

```

Node08 getSuccessor(Node08 del) {
    Node08 successor = del.right;
    Node08 successorParent = del;
    while (successor.left != null) {
        successorParent = successor;
        successor = successor.left;
    }
    if (successor != del.right) {

```

```

        successorParent.left = successor.right;
        successor.right = del.right;
    }
    return successor;
}

```

```

void delete(double ipk) {
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
        return;
    }
    //cari node (curent) yang akan dihapus
    Node08 parent = root;
    Node08 current = root;
    boolean isLeftChild = false;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            break;
        } else if (ipk < current.mahasiswa.ipk) {
            parent = current;
            current = current.left;
            isLeftChild = true;
        } else if (ipk > current.mahasiswa.ipk) {
            parent = current;
            current = current.right;
            isLeftChild = false;
        }
    }
    // penghapusan
    if (current == null) {

```

```

System.out.println("Data tidak ditemukan");

return;
} else {
    // jika tidak ada anak (leaf), maka node dihapus
    if (current.left == null && current.right == null) {
        if (current == root) {
            root = null;
        } else {
            if (isLeftChild) {
                parent.left = null;
            } else {
                parent.right = null;
            }
        }
    }
    } else if (current.left == null) { // jika hanya punya 1 anak (kanan)
        if (current == root) {
            root = current.right;
        } else {
            if (isLeftChild) {
                parent.left = current.right;
            } else {
                parent.right = current.right;
            }
        }
    }
    } else if (current.right == null) { // jika hanya punya 1 anak (kiri)
        if (current == root) {
            root = current.left;
        } else {
            if (isLeftChild) {
                parent.left = current.left;
            }
        }
    }
}

```



```

    } else {
        parent.right = current.left;
    }
}

} else if (current.right == null) { // jika punya 1 anak (kiri)
    if (current == root) {
        root = current.left;
    } else {
        if (isLeftChild) {
            parent.left = current.left;
        } else {
            parent.right = current.left;
        }
    }
} else {
    Node08 successor = getSuccessor(current);
    System.out.println("Jika 2 anak, current = ");
    successor.mahasiswa.tampilInformasi();
    if (current == root) {
        root = successor;
    } else {
        if (isLeftChild) {
            parent.left = successor;
        } else {
            parent.right = successor;
        }
    }
    successor.left = current.left;
}
}

```

```
}  
}
```

Kode Program BinaryTreeMain08 :

```
package PERTEMUAN14;
```

```
public class BinaryTreeMain08 {
```

```
    public static void main(String[] args) {
```

```
        BinaryTree08 bst = new BinaryTree08();
```

```
        bst.add(new Mahasiswa08 ("244160121", "Ali", "A", 3.57));
```

```
        bst.add(new Mahasiswa08 ("244160221", "Badar", "B", 3.85));
```

```
        bst.add(new Mahasiswa08 ("244160185", "Candra", "C", 3.21));
```

```
        bst.add(new Mahasiswa08 ("244160220", "Dewi", "B", 3.54));
```

```
        System.out.println("\nDaftar semua mahasiswa (in order traversal):");
```

```
        bst.traverseInOrder (bst.root);
```

```
        System.out.println("\nPencarian data mahasiswa:");
```

```
        System.out.print("Cari mahasiswa dengan ipk: 3.54: ");
```

```
        String hasilCari = bst.find(3.54)? "Ditemukan": "Tidak ditemukan";
```

```
        System.out.println(hasilCari);
```

```
        System.out.print("Cari mahasiswa dengan ipk: 3.22 ");
```

```
        hasilCari = bst.find(3.22)? "Ditemukan": "Tidak ditemukan";
```

```
        System.out.println(hasilCari);
```

```
        bst.add(new Mahasiswa08 ("244160131", "Devi", "A", 3.72));
```

```
        bst.add(new Mahasiswa08 ("244160205", "Ehsan", "D", 3.37));
```

```
        bst.add(new Mahasiswa08 ("244160170", "Fizi", "B", 3.46));
```

```

System.out.println("\nDaftar semua mahasiswa setelah penambahan 3 mahasiswa:");
System.out.println("InOrder Traversal:");
bst.traverseInOrder (bst.root);
System.out.println("\nPreOrder Traversal:");
bst.traversePreOrder (bst.root);
System.out.println("\nPostOrder Traversal:");
bst.traversePostOrder (bst.root);

System.out.println("\nPenghapusan data mahasiswa");
bst.delete(3.57);

System.out.println("\nDaftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order
traversal:");
bst.traverseInOrder (bst.root);
}
}

```

Pertanyaan Percobaan

1. Mengapa dalam binary search tree proses pencarian data bisa lebih efektif dilakukan dibanding binary tree biasa?

Jawab : Karena pada **Binary Search Tree (BST)** data disusun secara teratur.

Aturannya:

- Nilai lebih kecil → masuk ke kiri
- Nilai lebih besar → masuk ke kanan

Sehingga saat mencari data, kita tidak perlu mengecek semua node satu per satu.

Contoh:

Jika mencari IPK 3.54:

- jika nilai yang dicari lebih kecil → langsung ke kiri
- jika lebih besar → langsung ke kanan

Jadi pencarian lebih cepat dibanding binary tree biasa yang belum tentu teratur.

2. Untuk apakah di class Node, kegunaan dari atribut left dan right?

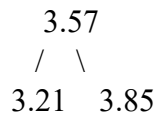
Jawab : Atribut:

Node08 left, right;

digunakan untuk menyimpan alamat/reference anak node.

- left → menunjuk anak kiri
- right → menunjuk anak kanan

Contoh:



- node 3.57 memiliki:
 - left → 3.21
 - right → 3.85

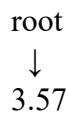
3. A. Untuk apakah kegunaan dari atribut root di dalam class BinaryTree?

Jawab : root digunakan untuk menyimpan node pertama atau akar dari tree.

Semua proses pada tree dimulai dari root, seperti:

- tambah data
- pencarian
- traversal
- delete

Contoh:



- B. Ketika objek tree pertama kali dibuat, apakah nilai dari root?

Jawab : Nilai root adalah:

null

Karena tree masih kosong dan belum memiliki node.

Pada constructor:

```
public BinaryTree08() {
    root = null;
}
```

4. Ketika tree masih kosong, dan akan ditambahkan sebuah node baru, proses apa yang akan terjadi?

Jawab : Saat tree kosong:

root == null

Maka node baru akan langsung menjadi root.

Kode:

```
if (isEmpty()) {
    root = newNode08;
}
```

Jadi node pertama otomatis menjadi akar tree.

5. Perhatikan method add(), di dalamnya terdapat baris program seperti di bawah ini. Jelaskan secara detil untuk apa baris program tersebut?

```
parent = current;
```

```

    if (mahasiswa.ipk < current.mahasiswa.ipk) {
        current = current.left;
        if (current == null) {
            parent.left = newNode08;
            return;
        }
    } else {
        current = current.right;
        if (current == null) {
            parent.right = newNode08;
            return;
        }
    }
}

```

Jawab :

Kode ini digunakan untuk menentukan posisi node baru pada Binary Search Tree.

parent = current;

Menyimpan posisi node sebelumnya.

Karena current akan berpindah ke kiri atau kanan, maka parent digunakan untuk mengingat node induknya.

if (mahasiswa.ipk < current.mahasiswa.ipk)

Mengecek apakah IPK baru lebih kecil dari node sekarang.

Jika lebih kecil:

→ pindah ke kiri.

current = current.left;

Pointer current bergerak ke anak kiri.

if (current == null)

Mengecek apakah posisi kiri kosong.

Jika kosong:

→ node baru ditempatkan di kiri parent.

parent.left = newNode08;

Menambahkan node baru sebagai anak kiri.

return;

Menghentikan proses add karena data sudah berhasil dimasukkan.

6. Jelaskan langkah-langkah pada method delete() saat menghapus sebuah node yang memiliki dua anak. Bagaimana method getSuccessor() membantu dalam proses ini?

Jawab :

Saat node memiliki 2 anak:

Contoh:

```

    3.57
   /  \
  3.21  3.85
   /
  3.72

```

Jika 3.57 dihapus:

tidak bisa langsung dihapus karena memiliki 2 anak.

Langkah-langkah delete:

1. Cari node yang akan dihapus

Menggunakan:

current

2. Cari successor

Successor adalah:

node paling kecil di subtree kanan.

Pada contoh:

- subtree kanan = 3.85
- node terkecil = 3.72

Jadi successor = 3.72

3. Method getSuccessor()

Kode:

```
Node08 successor = del.right;
```

```
while (successor.left != null) {  
    successor = successor.left;  
}
```

Digunakan untuk mencari node terkecil di bagian kanan.

4. Ganti node yang dihapus dengan successor

Misalnya:

3.57 dihapus

↓

diganti 3.72

Tree tetap memenuhi aturan BST.

Fungsi getSuccessor()

Method ini membantu:

- mencari pengganti node yang dihapus
- menjaga struktur BST tetap terurut
- mencegah tree rusak setelah delete

The screenshot shows an IDE window titled "Algoritma-dan-Struktur-Data". The code editor displays the following Java code:

```
package PERTEMUAN14;

public class BinaryTreeArrayMain08 {
    public static void main(String[] args) {
        BinaryTreeArray08 bta = new BinaryTreeArray08();
        Mahasiswa08 mhs1 = new Mahasiswa08 (nim: "244160121", nama: "Ali", kelas: "A", ipk: 3.57);
        Mahasiswa08 mhs2 = new Mahasiswa08 (nim: "244160185", nama: "Candra", kelas: "C", ipk: 3.41);
        Mahasiswa08 mhs3 = new Mahasiswa08 (nim: "244160221", nama: "Badar", kelas: "B", ipk: 3.75);
        Mahasiswa08 mhs4 = new Mahasiswa08 (nim: "244160220", nama: "Dewi", kelas: "B", ipk: 3.35);
        Mahasiswa08 mhs5 = new Mahasiswa08 (nim: "244160131", nama: "Devi", kelas: "A", ipk: 3.48);
        Mahasiswa08 mhs6 = new Mahasiswa08 (nim: "244160205", nama: "Ehsan", kelas: "D", ipk: 3.61);
        Mahasiswa08 mhs7 = new Mahasiswa08 (nim: "244160170", nama: "Fizi", kelas: "B", ipk: 3.86);

        Mahasiswa08[] dataMahasiswas = {mhs1, mhs2, mhs3, mhs4, mhs5, mhs6, mhs7};
        int idxLast = 6;
        bta.populateData (dataMahasiswas, idxLast);
        System.out.println(x: "\nInorder Traversal Mahasiswa: ");
        bta.traverseInOrder (idxStart: 0);
    }
}
```

The terminal output shows the result of the inorder traversal:

```
Inorder Traversal Mahasiswa:
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.35
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.41
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.48
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.61
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.75
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.86
PS D:\Algoritma-dan-Struktur-Data>
```

Kode Program BinaryTreeArray08 :

```
package PERTEMUAN14;
```

```
public class BinaryTreeArray08 {
```

```
    Mahasiswa08 [] dataMahasiswa;
```

```
    int idxLast;
```

```
    public BinaryTreeArray08 () {
```

```

this.dataMahasiswa = new Mahasiswa08 [10];
}

void populateData (Mahasiswa08 dataMhs[], int idxLast) {
this.dataMahasiswa = dataMhs;
this.idxLast = idxLast;
}

void traverseInOrder (int idxStart) {
if (idxStart <= idxLast) {
    if (dataMahasiswa [idxStart] != null) {
        traverseInOrder (2*idxStart+1);
        dataMahasiswa [idxStart].tampilInformasi();
        traverseInOrder (2*idxStart+2);
    }
}
}
}

```

Kode Program BinaryTreeArrayMain08 :

```

package PERTEMUAN14;

public class BinaryTreeArrayMain08 {
    public static void main(String[] args) {
        BinaryTreeArray08 bta = new BinaryTreeArray08();
        Mahasiswa08 mhs1 = new Mahasiswa08 ("244160121", "Ali", "A", 3.57);
        Mahasiswa08 mhs2 = new Mahasiswa08 ("244160185", "Candra", "C", 3.41);
        Mahasiswa08 mhs3= new Mahasiswa08 ("244160221", "Badar", "B", 3.75);
        Mahasiswa08 mhs4= new Mahasiswa08 ("244160220", "Dewi", "B", 3.35);
    }
}

```



```

Mahasiswa08 mhs5 = new Mahasiswa08 ("244160131", "Devi", "A", 3.48);
Mahasiswa08 mhs6 = new Mahasiswa08 ("244160205", "Ehsan", "D", 3.61);
Mahasiswa08 mhs7 = new Mahasiswa08 ("244160170", "Fizi", "B", 3.86);

Mahasiswa08[] dataMahasiswas = {mhs1, mhs2, mhs3, mhs4, mhs5, mhs6, mhs7, null,
null, null};

int idxLast = 6;

bta.populateData (dataMahasiswas,idxLast);

System.out.println("\nInorder Traversal Mahasiswa: ");

bta.traverseInOrder (0);

}

}

```

Pertanyaan Percobaan

1. Apakah kegunaan dari atribut data dan idxLast yang ada di class BinaryTreeArray?

Jawab : **Atribut dataMahasiswa**

Mahasiswa08[] dataMahasiswa;

Digunakan untuk menyimpan seluruh node binary tree dalam bentuk array.

Isi array:

index 0 = root

index 1 = anak kiri root

index 2 = anak kanan root

dan seterusnya

Jadi tree direpresentasikan menggunakan array.

Atribut idxLast

int idxLast;

Digunakan untuk menyimpan indeks terakhir yang berisi data pada array.

Tujuannya:

- mengetahui batas data
- agar traversal tidak melewati indeks terakhir

2. Apakah kegunaan dari method populateData()?

Jawab : Method:

void populateData(Mahasiswa08 dataMhs[], int idxLast)

digunakan untuk:

- mengisi array binary tree dengan data mahasiswa
- menentukan indeks terakhir data yang terisi

Kode:

```
this.dataMahasiswa = dataMhs;
```

```
this.idxLast = idxLast;
```

Artinya:

- array data disalin ke atribut class
- idxLast disimpan untuk traversal

3. Apakah kegunaan dari method `traverseInOrder()`?

Jawab : Method:

```
void traverseInOrder(int idxStart)
```

digunakan untuk menampilkan isi binary tree dengan urutan InOrder Traversal.

Urutan InOrder:

Kiri → Root → Kanan

Cara kerja:

```
traverseInOrder(2*idxStart+1);
```

→ traversal anak kiri

```
dataMahasiswa[idxStart].tampilInformasi();
```

→ tampilkan root

```
traverseInOrder(2*idxStart+2);
```

→ traversal anak kanan

4. Jika suatu node binary tree disimpan dalam array indeks 2, maka di indeks berapakah posisi left child dan right child masing-masing?

Jawab : Rumus binary tree array:

Left child

$2*i + 1$

Right child

$2*i + 2$

Jika node berada di indeks:

$i = 2$

Maka:

Left child

$2(2)+1 = 5$

Right child

$2(2)+2 = 6$

Jadi

- left child → indeks 5
- right child → indeks 6:

5. Apa kegunaan statement `int idxLast = 6` pada praktikum 2 percobaan nomor 4?

Jawab : Kode:

```
int idxLast = 6;
```

menandakan bahwa:

- indeks terakhir yang berisi data adalah indeks ke-6

Karena terdapat 7 data mahasiswa:

0 1 2 3 4 5 6

Traversal hanya dilakukan sampai indeks 6.

6. Mengapa indeks $2*idxStart+1$ dan $2*idxStart+2$ digunakan dalam pemanggilan rekursif, dan apa kaitannya dengan struktur pohon biner yang disusun dalam array?
Jawab : Karena binary tree disimpan dalam bentuk array.

TUGAS PRAKTIKUM

1. Buat method di dalam class BinaryTree00 yang akan menambahkan node dengan cara rekursif (addRekursif()).

Jawab :

```
public void addRekursif(Mahasiswa08 mahasiswa) {  
    root = addRekursif(root, mahasiswa);  
}  
  
private Node08 addRekursif(Node08 current, Mahasiswa08 mahasiswa) {  
  
    if (current == null) {  
        return new Node08(mahasiswa);  
    }  
  
    if (mahasiswa.ipk < current.mahasiswa.ipk) {  
        current.left = addRekursif(current.left, mahasiswa);  
    } else if (mahasiswa.ipk > current.mahasiswa.ipk) {  
        current.right = addRekursif(current.right, mahasiswa);  
    }  
  
    return current;  
}
```

2. Buat method di dalam class BinaryTree00 untuk menampilkan data mahasiswa dengan IPK paling kecil dan IPK yang paling besar (cariMinIPK() dan cariMaxIPK()) yang ada di dalam binary search tree.

Jawab :

```
public void cariMinIPK() {  
  
    if (isEmpty()) {  
        System.out.println("Tree kosong");  
        return;  
    }  
  
    Node08 current = root;  
  
    while (current.left != null) {  
        current = current.left;  
    }  
}
```

```

        System.out.println("Mahasiswa dengan IPK terkecil:");
        current.mahasiswa.tampilInformasi();
    }

    public void cariMaxIPK() {

        if (isEmpty()) {
            System.out.println("Tree kosong");
            return;
        }

        Node08 current = root;

        while (current.right != null) {
            current = current.right;
        }

        System.out.println("Mahasiswa dengan IPK terbesar:");
        current.mahasiswa.tampilInformasi();
    }
}

```

3. Buat method dalam class BinaryTree00 untuk menampilkan data mahasiswa dengan IPK di atas suatu batas tertentu, misal di atas 3.50 (tampilMahasiswaIPKdiAtas(double ipkBatas)) yang ada di dalam binary search tree.

Jawab :

```

public void tampilMahasiswaIPKdiAtas(Node08 node, double ipkBatas) {

    if (node != null) {

        tampilMahasiswaIPKdiAtas(node.left, ipkBatas);

        if (node.mahasiswa.ipk > ipkBatas) {
            node.mahasiswa.tampilInformasi();
        }

        tampilMahasiswaIPKdiAtas(node.right, ipkBatas);
    }
}

```

4. Modifikasi class BinaryTreeArray00 di atas, dan tambahkan : • method add(Mahasiswa data) untuk memasukkan data ke dalam binary tree • method traversePreOrder()

Jawab :

```

public void add(Mahasiswa08 data) {

```

```

        if (idxLast == dataMahasiswa.length - 1) {
            System.out.println("Array penuh");
            return;
        }

        idxLast++;
        dataMahasiswa[idxLast] = data;
    }

    void traversePreOrder(int idxStart) {

        if (idxStart <= idxLast) {

            if (dataMahasiswa[idxStart] != null) {

                dataMahasiswa[idxStart].tampilInformasi();

                traversePreOrder(2 * idxStart + 1);

                traversePreOrder(2 * idxStart + 2);
            }
        }
    }
}

```

The screenshot shows a Java IDE with the following code:

```

// BinaryTreeArrayMahasiswa.java
import java.util.*;

public class BinaryTreeArrayMahasiswa {
    private static ArrayList<Mahasiswa> dataMahasiswa = new ArrayList<>();
    private static int idxLast = -1;

    public static void main(String[] args) {
        Mahasiswa m1 = new Mahasiswa(1001, "Rendi", "A", 85.0);
        Mahasiswa m2 = new Mahasiswa(1002, "Candra", "B", 80.0);
        Mahasiswa m3 = new Mahasiswa(1003, "Dewi", "A", 88.0);
        Mahasiswa m4 = new Mahasiswa(1004, "Ali", "B", 82.0);
        Mahasiswa m5 = new Mahasiswa(1005, "Rendi", "A", 85.0);
        Mahasiswa m6 = new Mahasiswa(1006, "Candra", "B", 80.0);
        Mahasiswa m7 = new Mahasiswa(1007, "Dewi", "A", 88.0);
        Mahasiswa m8 = new Mahasiswa(1008, "Ali", "B", 82.0);
        Mahasiswa m9 = new Mahasiswa(1009, "Rendi", "A", 85.0);
        Mahasiswa m10 = new Mahasiswa(1010, "Candra", "B", 80.0);

        dataMahasiswa.add(m1);
        dataMahasiswa.add(m2);
        dataMahasiswa.add(m3);
        dataMahasiswa.add(m4);
        dataMahasiswa.add(m5);
        dataMahasiswa.add(m6);
        dataMahasiswa.add(m7);
        dataMahasiswa.add(m8);
        dataMahasiswa.add(m9);
        dataMahasiswa.add(m10);

        traversePreOrder(0);
    }

    private static void traversePreOrder(int idxStart) {
        if (idxStart <= idxLast) {
            if (dataMahasiswa[idxStart] != null) {
                dataMahasiswa[idxStart].tampilInformasi();
                traversePreOrder(2 * idxStart + 1);
                traversePreOrder(2 * idxStart + 2);
            }
        }
    }
}

```

The output of the program is as follows:

```

PS D:\Algoritma dan Struktur Data> java BinaryTreeArrayMahasiswa
1001 Rendi A 85.0
1002 Candra B 80.0
1003 Dewi A 88.0
1004 Ali B 82.0
1005 Rendi A 85.0
1006 Candra B 80.0
1007 Dewi A 88.0
1008 Ali B 82.0
1009 Rendi A 85.0
1010 Candra B 80.0
PS D:\Algoritma dan Struktur Data>

```